Proyecto Diseño

Samuel Zuluaga -202520012
Tulio Robinson -202523681
Miguel Suarez- 202520019

Tabla de Contenido.

1. Contexto **2**
1.1 Objetivos 2
1.2 No objetivos 3
1.3 RF: 3
1.4 RNF: 6
1.5 Restricciones 6
2. Componentes **8**
2.1 Diagrama de contexto 8
2.2 Componentes 8
3. Roles **9**
3.1 Clase, estereotipo y descripción. 9
4. Responsabilidades **13**
4.1 Tabla de Responsabilidades 13
4.2 Justificación 19
5. Colaboraciones **20**
5.1 Funcionalidades críticas **20**
6. Control **24**
6.1. Estilo de control 24
7. Alto Nivel **25**
7.1 Clases y relaciones 25
8. Detallado **25**
8.1 Atributos, métodos y cardinalidad. 25
8.2 Descripción de cada clase 25
9. Decision **34**
9.1 Sección 1. Contexto 34
9.2 Sección 2. Componentes 35
9.3 Sección 3. Roles 35
9.4 Sección 4. Responsabilidades 36
10. Documentación IAG. **36**

1. Contexto

1.1 Objetivos

Dividimos el enfoque del problema y del sistema en 4 objetivos relevantes.

1. Objetivo 1.
   Implementar en Java la lógica para los cinco tipos de usuarios, para que cada requisito funcional pueda ser demostrado mediante su debida prueba por consola con datos de ejemplo.
2. Objetivo 2.
   Poder cumplir con las reglas del negocio; como las sanciones por tarjetas, el mínimo del personal disponible, el IVA, los descuentos y los puntos de fidelidad de los socios.
3. Objetivo 3.
   Mantener coherencia con la parte de análisis hecha (modelo de dominio, historias de usuario y UML), de modo que todo cambio parta de la misma base sin reemplazar la solución propuesta inicialmente, y que cada cambio quede registrado y debidamente justificado.
4. Objetivo 4.
   Una arquitectura de diseño efectiva y fácil de actualizar. Siguiendo el modelo MVC (Modelo, Vista, Controlador) usando la consola como vista, la lógica del modelo siendo como tal el sistema dividido por subpaquetes por el dominio propuesto (usuarios, deportes y tienda) y los gestores como controlador. Teniendo aparte la persistencia, para que quede separada, y cada cambio toque solo el paquete que le corresponde.

1.2 No objetivos

1. Interfaz de interacción con el usuario: cosas como menús, validación de entradas y presentación de resultados o reportes quedan para un futuro. En esta entrega mostraremos casos de prueba por consola para verificar los requerimientos funcionales propuestos.
2. Actualización del ranking de los jugadores. Cada jugador conserva un puntaje en el modelo, pero después de cada partido no vamos a actualizar ni recalcular este puntaje.

3. Control de acceso físico a las canchas por bebidas calientes. El enunciado pedía prohibir que una persona que haya comprado una bebida caliente entre a una instalación de tenis de mesa techada o cancha de pádel, el sistema no tiene cómo saber que lleva consigo una persona. Por esto, la regla se verifica en los momentos en los que el sistema si tiene la información, cuando se le vende la bebida caliente se le rechaza si tiene una reserva vigente en una mesa de tenis techada o cancha de pádel, y al reservar una de estas se rechaza si el socio compró una bebida caliente dentro del intervalo de tiempo decidido por la restricción.
4. Cobro de mensualidades: no se describe cómo se cobra una mensualidad, por eso no lo vamos a modelar. Sin embargo puede discriminarse como un posible rubro por si este cobro se implementa más adelante.
5. Seguridad avanzada en las contraseñas. Cada usuario debe tener su login y contraseña, pero no se modelan cifrado, recuperación ni bloqueo por intentos fallidos. La contraseña se almacena y se lee solamente por la aplicación y se guarda en la carpeta de datos (como mencionado en el enunciado), ya que ningún usuario malicioso podrá modificar estos archivos.

1.3 RF:

1. Sistema

RF1: El sistema debe guardar todos los cambios en la carpeta de datos y recuperarlos al iniciar.

RF2: Un socio solo puede consultar su propia información.

2. Usuario.

RF1: Cada usuario debe poder acceder a su perfil digitando su login y password.

3. Socio.

RF1: Cada socio debe poder ver cual es la posición asignada para el próximo partido

RF2: Cada socio debe poder ver cuál es su estado para el próximo partido, es decir si se encuentra suspendido. En caso de estar suspendido debe poder consultar hasta cuándo lo estará.

RF3: Cada socio debe poder crear nuevos partidos contra otros socios de su mismo nivel o adyacente, o en contra de un entrenador. En caso de manifestarse alguna inconsistencia, el sistema debe rechazar la creación de este partido y referenciar el motivo de porque no se puede crear.

RF4: Cada socio debe poder reservar instalaciones indicando el horario, jugadores y modalidad. En caso de estar lleno, el sistema debe rechazar la reserva.

RF5: El sistema debe rechazar la compra de bebidas calientes a los socios mientras estos tengan reserva activa en pádel o tenis de mesa techado, y debe rechazar la reserva de esas instalaciones a un socio que compró una bebida caliente dentro del intervalo definido en las restricciones.

RF6: Cada socio debe poder comprar varios artículos (con existencia en inventario previamente consultado) en una sola transacción, la cual ya incluye su IVA correspondiente (19%).

RF7: Siempre que se compre un snack se le debe recordar al socio los alérgenos de este mismo.

RF8: Toda venta de cafetería debe incluir en su costo el impuesto al consumo 8% y ofrece una propina sugerida de 10%, que el socio puede modificar.

RF9: Toda venta en general, genera para el comprador puntos de fidelidad equivalentes al 2% de su valor (1 punto = 1 peso). El socio puede redimirlos como descuentos.

RF10: Los socios deben poder usar el 8% de descuento que les haya compartido un empleado

RF11: Los socios deben poder inscribirse ya sea a una disciplina individual o un equipo.

RF12: Los socios deben poder acceder a su información personal, como asistencia, evaluaciones y estadísticas de desempeño.

4. Empleado.

RF1: El empleado debe poder solicitar un cambio de turno. En caso de ya haber pasado la fecha para la que lo solicita el sistema debe rechazar la solicitud.

RF2: El empleado debe poder proponer productos. El sistema debe permitir subir estos.

RF3: El empleado debe poder comprar con su código de descuento (15% para él)

RF4: El sistema debe permitir a los empleados compartir su código de descuento con los socios.

5. Administrador:

RF1: Debe poder rechazar o aceptar solicitudes de cambio de turno. El sistema rechaza automáticamente la solicitud si quedan menos de 2 entrenadores o menos de 1 fisioterapeuta disponibles.
RF2: El sistema debe permitir que los administradores aprueben o rechacen las sugerencias de los empleados.
RF3: Debe poder consultar la información de las ventas, por rubro (tienda, cafetería, mensualidades) y por periodo.
RF4: Los administradores deben poder acceder a los productos presentes en el inventario y consultar toda la información de estos.
RF5: Los administradores deben poder actualizar las cifras que están relacionadas al inventario.
RF6. Deben tener la posibilidad de crear nuevos equipos y eliminarlos. En caso de crear un equipo ya existente, el sistema debe rechazar la solicitud de creación. Si se intenta borrar un equipo no existente, el sistema debe rechazar la solicitud.
RF7: Deben tener la posibilidad de añadir o quitar jugadores a los equipos. En caso de añadir a un jugador ya presente en un equipo se debe rechazar la solicitud y en caso de tratar un jugador no existente también se debe rechazar la solicitud.
RF8: Debe poder administrar los turnos de los empleados.
RF9: Debe poder consultar el historial de reservas, partidos y sesiones de entrenamiento.

6. Entrenador:

RF1: Debe poder crear una sesión de entrenamiento donde pueda incluir la fecha, hora, duración, instalación y convocados. En caso de estar ocupada la instalación el sistema debe rechazar la solicitud de creación.
RF2: Al finalizar cada sesión el entrenador debe poder registrar para cada convocado la asistencia, y opcionalmente la nota de 1 a 5.
RF3: Debe poder registrar los jugadores convocados para un partido, el sistema debe validar que cumpla con el mínimo de jugadores habilitados para el deporte y que junto a esto no haya jugadores suspendidos.
RF4: A la hora de un entrenador inscribir un equipo a una competencia, el sistema debe validar que todos los jugadores pertenezcan a la categoría permitida y a la nómina mínima. En caso de haber algún problema, debe rechazar la inscripción indicando que el jugador no está pasando algún filtro.
RF5: El entrenador debe poder acceder a las estadísticas de cada miembro de los equipos que maneja.

RF6: Debe poder registrar el resultado y las estadísticas de un partido de competencia.

1.4 RNF:

1. Consistencia con los datos:
   Si una operación se rechaza, se debe cancelar todo el flujo de esta, osea una compra rechazada no descuenta inventario ni suma puntos.
2. Mensajes de rechazo informativos:
   Todo rechazo de acción o operación debe indicar el motivo.
3. Modificabilidad de la lógica:
   La lógica no depende de la consola ni de la persistencia, por eso se modelan como paquetes separados, para que así se pueda cambiar la interfaz o otras cosas fácilmente sin cambiar las reglas del enunciado y el negocio.
4. Verificación de Requerimientos funcionales.
   Cada requerimiento funcional debe ser verificado y demostrado (probado) por la consola con sus casos de éxito y también rechazo.

1.5 Restricciones

R1: Cada jugador solo puede pertenecer a un equipo de cada deporte.

R2: Los partidos internos solo se pueden realizar entre equipos de la misma categoría y deporte.

R3: Cada equipo debe cumplir con el mínimo de jugadores.

R4: Un jugador solo se puede vincular a un equipo si su edad corresponde a la categoría.

R5: Se permite un máximo de instalaciones usadas al mismo tiempo por cada disciplina.

R6: El tenis o tenis de mesa tienen capacidad de 2 o 4 personas,mientras el pádel de 4.

R7: Siempre debe haber disponible 2 entrenadores y un fisioterapeuta como mínimo.

R8: El administrador no tiene permiso de inscribirse a partidos ni de reservar instalaciones.

R9: Dos tarjetas amarillas o una roja suspenden al jugador. La acumulación de tarjetas se lleva a lo largo de toda la competencia.

R10: Solo los jugadores convocados y habilitados pueden jugar en una competencia, y un jugador suspendido no puede ser alineado en sus partidos.

R11: El número de camiseta de un jugador no puede repetirse dentro de su equipo.

R12: Las competencias tienen formato de liga (todos contra todos, con tablas de posiciones) o por fases (Fase de grupos - eliminación directa). En la liga los puntos son independientes por

deporte, en fútbol la victoria son 3 puntos, derrota 0, empate 1. En baloncesto y voleibol no hay empates.

R13: La asistencia a un entrenamiento es dividida en asistio, falto o llego tarde. La evaluación del socio es opcional por el entrenador, con nota de 1 a 5 y comentario.

R14: No se vende una bebida caliente a un socio con reserva vigente en cancha de pádel o tenis de mesa techada, ni se reservan esas instalaciones a un socio que compró una bebida caliente en los últimos 30 minutos.

R15: Todo snack tiene registrados sus posibles alérgenos (puede no tener ninguno).

R16: Tarifas.

- Iva 19%
- Impuesto al consumo 8%
- Propina sugerida 10%
- Puntos de fidelidad 2% del valor de cada venta.
- Descuentos de empleados. 15% para el empleado y 8% para el socio con quien lo comparte.

R17: La aplicación se desarrolla en java.

R18: La información se almacena en archivos JSON dentro de una carpeta de datos, que no puede ser la misma donde está el código fuente.

R19: Todos los usuarios del sistema tienen un login y una contraseña.

R20: Se asume que solo la aplicación lee y escribe en la carpeta de datos, y que ningún usuario puede modificar esos archivos por fuera de ella.

2. Componentes

2.1 Diagrama de contexto

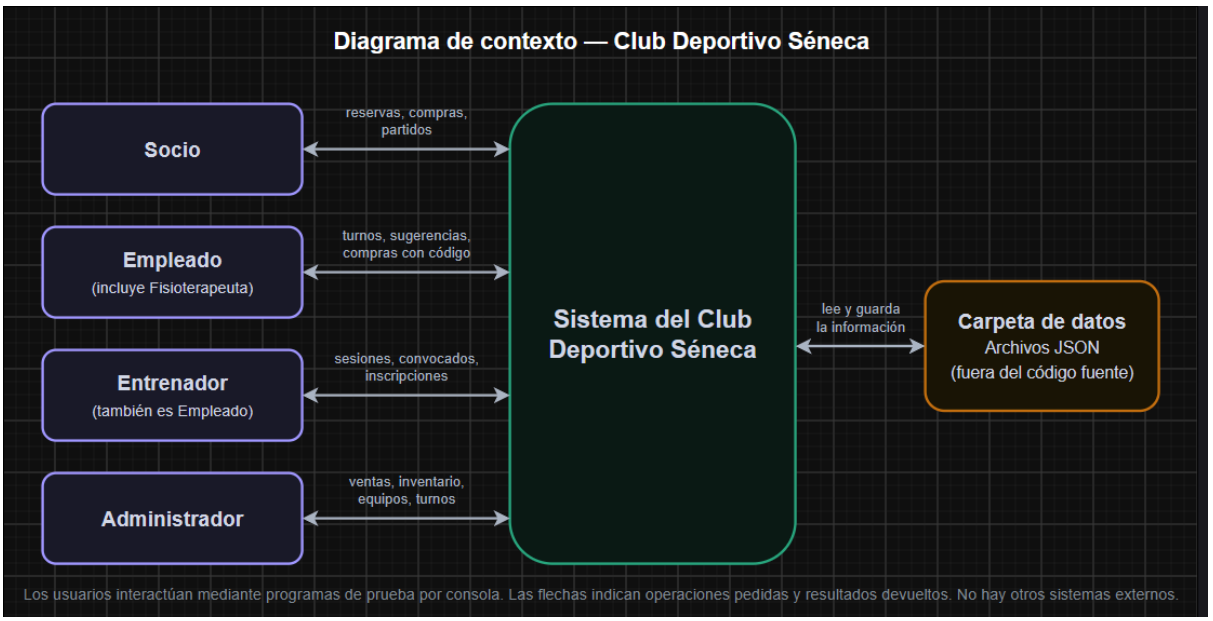


Figura 1. Diagrama de contexto

Se muestra el diagrama general de contexto en donde el sistema recibe las operaciones de los cuatro tipos de usuario, y devuelve sus resultados. Toda esta información se guarda en archivos JSON, dentro de una carpeta separada a la del código fuente. Sistema del Club Deportivo Séneca es la caja del sistema que se diagrama con más profundidad en el inciso 2.2

2.2 Componentes

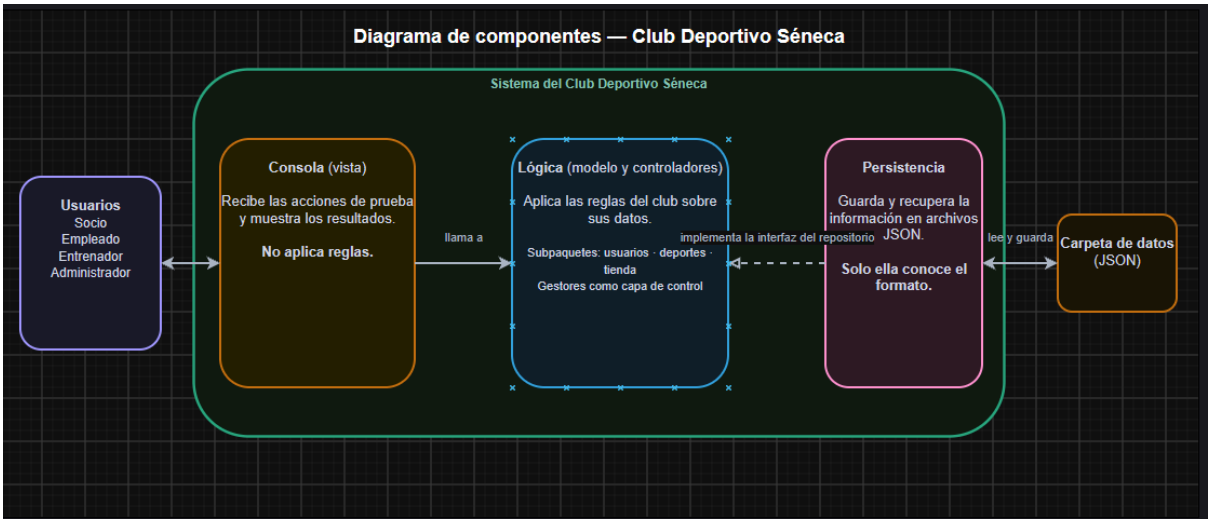


Figura 2. Diagrama de componentes.

Decidimos dividirlo en tres componentes, siguiendo los objetivos propuestos.

- Consola: Comprueba casos para verificar requerimientos funcionales y muestra los resultados, sin aplicar ninguna regla del enunciado.
- Lógica: Aplica las reglas del club y se organiza en subpaquetes (usuarios, deportes y tienda).
- Persistencia: Guarda la información y la recupera en archivos JSON.

La consola llama a la lógica, pero la lógica no conoce la consola. Adicionalmente, la lógica define una interfaz que la persistencia implementa, por lo que la lógica no conoce el formato de los archivos.

3. Roles

3.1 Clase, estereotipo y descripción.

Usuarios.

Candidato	Descripción	Estereotipo
Usuario	Persona con login, password y puntos de fidelidad	Information holder
Socio	Usuario del club con edad, que se inscribe, reserva y compra	Information holder
Empleado	Usuarios con turnos y código de descuento.	Information holder
Administrador	Usuarios con permisos administrativos.	Information holder
Entrenador	Empleado que dirige equipos y sesiones.	Information holder
Fisioterapeuta	Empleado que cuenta con el mínimo de personal por turno.	Information holder
Turno	Intervalo de horario de trabajo de un empleado.	Information holder
SolicitudCambioTurno	Petición de cambio de un	Information holder

	turno, con su estado	

Deportes.

Candidato	Descripción	Estereotipo
DeporteConjunto	Futbol, baloncesto o voleibol.	Information holder
Equipo	Agrupa a los jugadores en un deporte y categoría, con cupo limitado.	Structurer
InscripcionEquipo	Vincula a un socio con un equipo.	Information holder
Competencia	Torneo con formato (liga o fases) que organiza partidos entre equipos	Structurer
InscripcionCompetencia	Vincula a un equipo con una competencia,	Structurer
Partido	Registra fecha y marcador de un partido con sus estadísticas,	Structurer
PartidoInterno	Partido entre dos equipos del club,	Structurer
PartidoCompetencia	Partido oficial con rival.	Structurer
Estadistica	Cifras de un jugador en un partido: goles, sets, puntos, tarjetas.	Information holder
DisciplinaIndividual	Tenis, padel o tenis de mesa.	Information holder
InscripcionDisciplina	Vincula a un socio con una disciplina	Information holder
Instalación	Cancha de padel o mesa de tenis de mesa.	Information holder
Reserva	Uso de una instalación en un horario.	Information holder
PartidoPractica	Partido entre socios de nivel igual o adyacente.	Structurer

SesionEntrenamiento	Entrenamiento con fecha, instalación, entrenador y convocados.	Structurer
Asistencia	Estado de un convocado	Information holder

Tienda.

Candidato	Descripción	Estereotipo
Producto	Producto con nombre y precio	Information holder
Articulo	Producto de tienda, con categoría y existencias.	Information holder
Bebida	Producto de cafetería, fría o caliente.	Information holder
Snack	Producto de cafetería con sus alérgenos.	Information holder
Venta	Agrupa las líneas de una compra y calcula su total.	Structurer
VentaTienda	Venta de tienda, con IVA	Structurer
VentaCafeteria	Venta de cafetería, con impuesto al consumo y propina.	Structurer
LineaVenta	Cantidad de un producto dentro de una venta	Information holder
Sugerencia	Producto propuesto por un empleado, con su estado	Information holder

Clases de aplicación.

Candidato	Descripción	Estereotipo
GestorUsuarios	Verifica el login y entrega el perfil de cada usuario	Controller
GestorEquipos	Crea equipo y valida la entrada de jugadores	Controller

GestorCompetencias	Válida inscripciones, convocados y suspensiones en competencia	Controller
GestorReservas	Valida y registra reservas y partidos de práctica	Controller
GestorEntrenamientos	Programa sesiones y registra asistencia y evaluación	Controller
GestorVentas	Valida y registra ventas	Controller
GestorInventario	Controla existencias y sugerencias de productos	Controller
GestorTurnos	Decide los cambios de turno según el mínimo de personal	Controller
RepositorioDatos (interfaz)	Define como la lógica pide guardar y cargar datos	Interfacer
RepositorioJSON	Implementa el repositorio con archivos JSON	Interfacer
Consola	Ejecutan los casos de éxito y rechazo de los RF	Interfacer
Club	Crea los gestores y el repositorio y los conecta entre sí.	Coordinator

Candidatos descartados.

Los candidatos descartados los relacionamos con clases que por el enunciado se podrían entender como candidatos claros para estas tablas.

- Compra: No lo hicimos como clase aparte, porque ya se modela mediante VentaTienda y VentaCafeteria.
- CodigoDescuento: Es un atributo de empleado, y mediante la asociación entre empleado y socio se le comparte a este.
- PuntosFidelidad: Se diseña como atributo de Usuario.
- Ranking: Entra como atributo puntajeRanking de InscripcionDisciplina, ya que decidimos que el sistema no recalcula.
- Sanción: Son atributos de EstadisticaFutbol.

- Mensualidad: No se modela, por decisión propia y porque el caso no describe cómo se cobra.

4. Responsabilidades

4.1 Tabla de Responsabilidades

Usuarios y Acceso.

#	Responsabilidad	Tipo	Asignada a
1	Guardar y entregar sus propios datos	Saber	Cada clase information holder
2	Verificar login y password	Decidir	GestorUsuarios
3	Decidir si el rol puede ejecutar una acción.	Decidir	Usuario y sus subclases
4	Entregar a cada socio solo su propia información	Hacer	GestorUsuarios
5	Acumular puntos y rechazar la redención si no alcanzan	Decidir	Usuario
6	Registrar que un empleado comparte su codigo con un socio.	Hacer	GestorUsuarios
7	Saber si el socio tiene un código compartido y de qué empleado	Saber	Socio

Equipos y competencias.

#	Responsabilidad	Tipo	Asignada a
8	Decidir si un socio entra al equipo	Decidir	Equipo
9	Decidir que un socio no esté en otro equipo del mismo deporte	Decidir	GestorEquipos
10	Crear y eliminar equipos, también añadir y quitar jugadores	Hacer	GestorEquipos
11	Consultar las estadísticas de los miembros los equipos de un entrenador	Hacer	GestorEquipos
12	Carga las competencias del club (nombre, organizador, temporada, categoría, formato) al iniciar y entregarlas.	Hacer	GestorCompetencias
13	Decidir si la categoria del equipo es valida para la competencia	Decidir	Competencia
14	Saber la nómina mínima de la competencia.	Saber	Competencia
15	Confirmar que el equipo cumple la nómina y que cada jugador pasa el filtro.	Decidir	Equipo
16	Crear y guardar la inscripción del equipo en competencia	Hacer	InscripcionCompetencia

17	Saber el mínimo de jugadores de un deporte	Saber	DeporteConjunto
18	Decidir si un socio está suspendido	Decidir	Socio
19	Confirmar que la alineación cumple el mínimo y no tiene suspendidos	Decidir	Equipo
20	Registrar la alineación en el partido	Hacer	Partido
21	Registrar el resultado y las estadísticas de un partido de competencia	Hacer	GestorCompetencias
22	Guardar las tarjetas amarillas y rojas de cada jugador	Saber	EstadisticaFutbol
23	Calcular hasta cuando dura la suspensión	Hacer	Socio
24	Informar al socio su posición y su estado para el proximo partido	Hacer	GestorCompetencias
25	Consultar el historial de partidos	Hacer	GestorCompetencias
26	Calcular los puntos de un partido de liga.	Hacer	Competencia

Deportes Individuales.

#	Responsabilidad	Tipo	Asignada a
27	Decidir si dos	Decidir	InscripcionDisciplin

	niveles son iguales o adyacentes		a
28	Inscribir a un socio en una disciplina	Hacer	GestorReservas
29	Decidir si los jugadores y la modalidad caben en la instalación	Decidir	Instalación
30	Recibir la reserva y consultar si el usuario puede reservar, decidir el cupo simultáneo y registrarla	Decidir	GestorReservas
31	Decidir si se crea un partido de práctica	Decidir	GestorReservas
32	Rechazar la reserva de pádel o tenis de mesa techada, si compro bebida caliente en el intervalo	Decidir	GestorReservas
33	Informar si un socio tiene una reserva vigente en las instalaciones	Hacer	GestorReservas
34	Consultar el historial de reservas y partidos de practica	Hacer	GestorReservas

Entrenamiento.

#	Responsabilidad	Tipo	Asignada a
35	Decidir si se puede crear una sesión	Decidir	GestorEntrenamientos
36	Registrar asistencia y nota de cada convocado	Hacer	SesionEntrenamiento
37	Rechazar una nota	Decidir	Asistencia

	fuera del rango 1 a 5		
38	Consultar el historial de sesiones	Hacer	GestorEntrenamientos

Tiendas y ventas.

#	Responsabilidad	Tipo	Asignada a
39	Informar la cantidad disponible, descontar existencias y rechazar si no alcanzan.	Decidir	Articulo
40	Validar la disponibilidad de todos los artículos antes de descontar	Decidir	GestorVentas
41	Agrupar las líneas de una venta	Hacer	Venta
42	Calcular descuento, iva o impuesto al consumo con propina y el total	Hacer	Venta
43	Calcular los puntos (2%) y pedir que se le acrediten al socio	Hacer	GestorVentas
44	Rechazar la bebida caliente si el socio tiene reserva vigente	Decidir	GestorVentas
45	Informar los alergenos de un snack	Hacer	GestorVentas
46	Consultar ventas por rubro y periodo	Hacer	GestorVentas
47	Consultar el inventario, reabastecer y mover articulos	Hacer	GestorInventario

48	Registrar una sugerencia de un empleado	Hacer	GestorInventario
49	Aprobar o rechazar una sugerencia para los productos	Decidir	GestorInventario

Turnos.

#	Responsabilidad	Tipo	Asignada a
50	Rechazar una solicitud de cambio con fecha pasada	Decidir	GestorTurnos
51	Decidir si se aprueba (siempre al menos 2 entrenadores y 1 fisioterapeuta)	Decidir	GestorTurnos
52	Crear, modificar y consultar los turnos	Hacer	GestorTurnos

Infraestructura

#	Responsabilidad	Tipo	Asignada a
53	Guardar y recuperar la información en JSON	Hacer	RepositorioJSON

54	Definir las operaciones de guardar y cargar	Saber	RepositorioDatos
55	Crear los gestores y entregarles el repositorio	Hacer	Club
56	Pedir acciones a los gestores y mostrar resultados y rechazos.	Hacer	Consola

4.2 Justificación

Justificamos las responsabilidades más complejas.

#3 Permisos por rol. Cada subclase de Usuario responde con métodos, para verificar que acciones tiene permitidas y quien las ejecuta. Para que se conserve la consistencia con la restricción R8.

#8 y #9 Equipo y gestor de equipos. Equipo decide lo que conoce de sí mismo: categoría, cupo y camisetas. Pero GestorEquipos ve todo lo de los otros equipos y verifica que un jugador no esté ya en otro equipo del mismo deporte (R1).

#13, #16, #19 y #20. Inscripción y alineación. Competencia sabe su categoría y su nómina mínima, DeporteConjunto el mínimo de jugadores, Socio si está suspendido. Equipo reúne esas respuestas y pide el registro.

#18, #22 y #23 Tarjetas y suspensión. EstadisticaFutbol guarda las tarjetas de cada partido. Socio es el que decide si está suspendido sumando todas las tarjetas de la competencia. GestorCompetencias sólo registra las estadísticas y no decide la sanción.

#32, #33 y #44 Bebida Caliente. La regla cruza las ventas y las reservas. GestorVentas y Gestor Reservas se consultan entre sí, y cada uno decide con lo que conoce.

#55 Club. Crea los gestores y les entrega el repositorio, para que la lógica esté aislada del RepositorioJson.

5. Colaboraciones

5.1 Funcionalidades críticas

Se detectaron las siguientes funcionalidades críticas:

a. Realizar login y verificar rol del usuario (RF1 de usuario)

Se toma esta como funcionalidad crítica debido a su importancia a nivel del sistema. Verificar el rol permite que se cumpla con las restricciones de cada usuario, pues después de la validación en el sistema, cada usuario podrá solo realizar las acciones que tiene permitidas según su rol. Es decir, que este requerimiento permite diferenciar el comportamiento de cada usuario por su rol.

En este caso, se muestra la secuencia de un caso en específico (administrador intenta reservar), sin embargo, esta secuencia se va a repetir en todos los métodos que necesiten validación de rol, como también estan: puedeRegistrarEquipo(), puedeAprobarTurno(), entre otros. El objetivo es que en las clases de usuario y sus respectivas subclases, se definan como métodos los permisos de cada usuario. Cuando se ejecute el método, se realizará una validación con uso de If donde se verificará a través de los métodos previamente implementados si el usuario puede realizar la acción o no.

Quien recibe la solicitud: la clase que contiene el método que se va a realizar, en el ejemplo, la recibe la clase GestorReservas.

Quien decide: la clase de usuario con el método puedeX(), en el ejemplo, la clase administrador con el método puedeReservar().

Quien delega: La clase que contiene el método delega a la clase de usuario la consulta de si tiene permiso y se comporta según la respuesta que reciba, en el ejemplo, GestorReservas le pregunta a Administrador y actúa según la respuesta que reciba.

Diagrama de secuencia (Administrador intenta reservar):

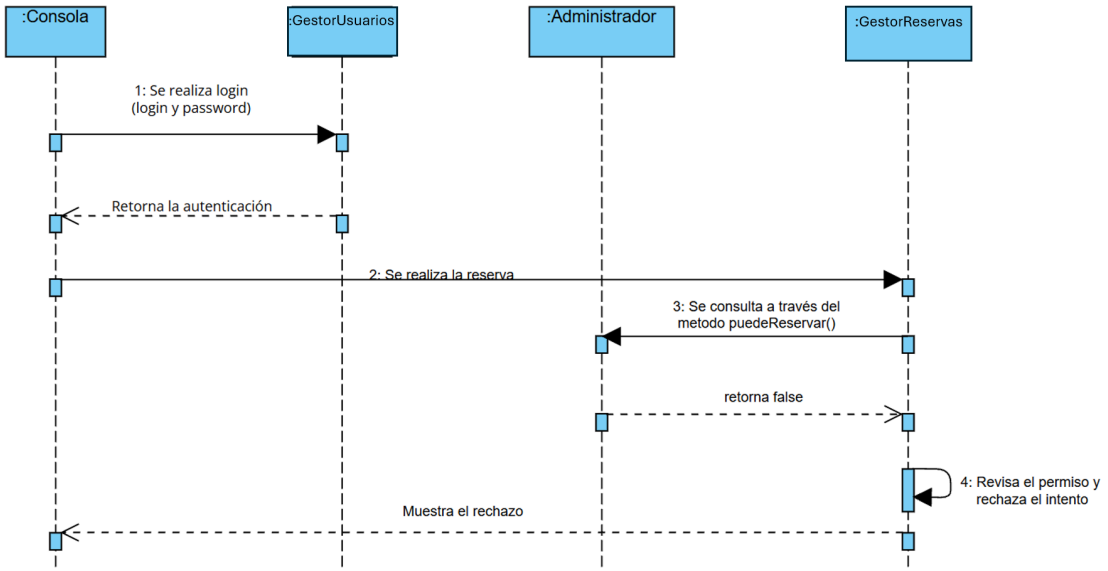


Figura 3. Diagrama de secuencia (Administrador intenta reservar).

b. Inscribir los equipos en una competencia (RF4 de entrenador)

Se seleccionó como una funcionalidad crítica debido a su complejidad, ya que por las restricciones establecidas es necesario realizar múltiples validaciones para que la inscripción se pueda cumplir. Para esto el equipo debe pertenecer a la categoría habilitada, debe cumplir con la nómina mínima de jugadores y todos los jugadores deben pertenecer al equipo. Es decir, son 3 validaciones necesarias para poder inscribir un equipo, esto demuestra la complejidad que la cataloga como una funcionalidad crítica.

Quien recibe la solicitud: La clase equipo recibe la solicitud de inscripción

Quien decide: La decisión es la unión entre datos de Equipo y Competencia, donde la competencia decide si coincide la categoría y cantidad mínima de jugadores, mientras que el equipo decide si coinciden los jugadores del equipo.

A quién delega: Equipo delega a la clase Competencia si cumple con categoría y nómina y delega en la clase InscripcionCompetencia la creación del registro del equipo.

Diagrama de secuencia:

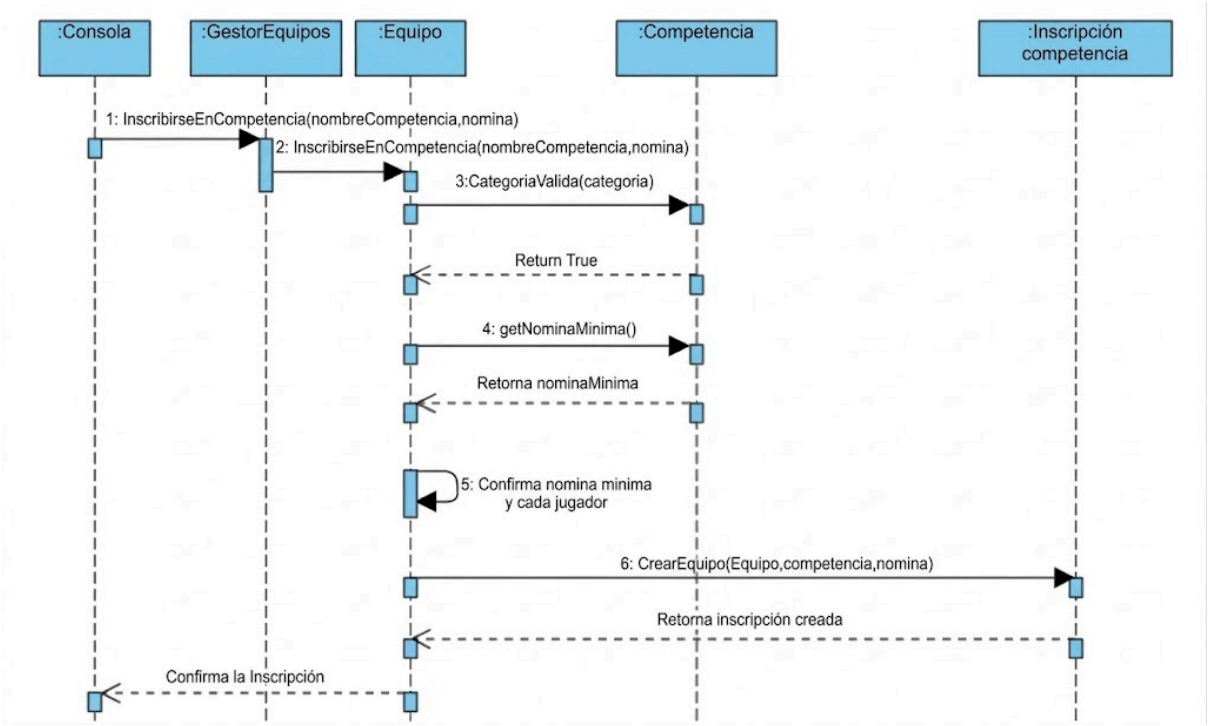


Figura 4. Diagrama de secuencia (Inscribirse en competencia).

c. Alinear jugadores a un partido (RF3 de entrenador):

Se considera una funcionalidad crítica debido a su complejidad pues se deben hacer dos validaciones para poder realizar la alineación. La primera evaluación es verificar la cantidad de jugadores con tal de que cumplan la nómina mínima del deporte que están jugando y la segunda evaluación es verificar que todos los jugadores estén habilitados para jugar.

Quién recibe la solicitud: La solicitud la recibe el equipo, al cual se le pide alinear ciertos jugadores según la plantilla que posea.

Quién decide: La decisión es tomada por las clases DeporteConjunto y Socio, donde el deporte conjunto determina el mínimo de jugadores para el deporte y socio determina las sanciones de cada miembro del equipo.

A quien delega: Equipo le pide a DeporteConjunto el mínimo de jugadores; a Socio le consulta las sanciones de cada miembro del equipo y finalmente ordena a partido el registro de la alineación.

Diagrama de secuencia:

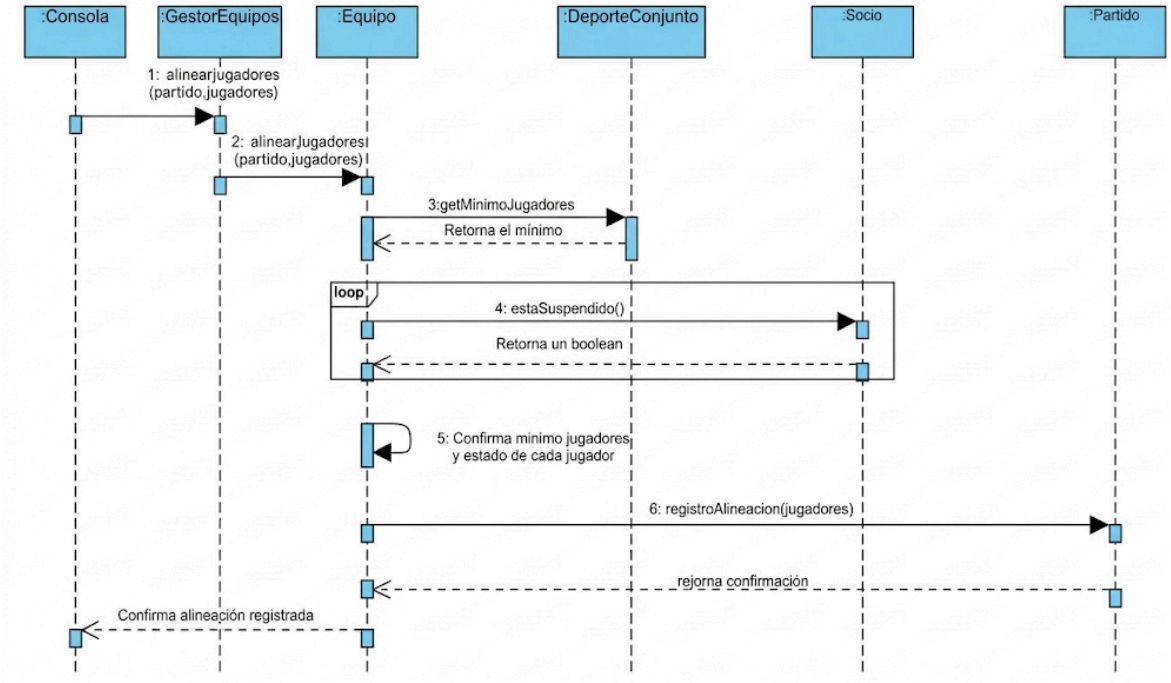


Figura 5. Diagrama de secuencia (Alinear jugadores).

d. Comprar en tienda (RF6, RF9 y RF10 de socio)

Esta funcionalidad se determina como crítica ya que el valor de la compra depende de múltiples acciones y el conjunto de estas. En este caso es necesario: validar el inventario disponible; aplicar el IVA del 19% a la compra; en caso de tener código de trabajador, aplicar el 8% de descuento y calcular puntos de fidelidad que equivalen al 2% de la compra. Cómo son múltiples procesos que se tienen que realizar para llegar al total de la compra, se tiene en cuenta esta funcionalidad como una crítica.

Quien recibe la solicitud: La clase que recibe la solicitud es GestorVentas, a la cual un socio le pide comprar determinada cantidad de productos.

Quien decide: Esta decisión es conjunta. En este caso Artículo determina si hay stock de este mismo, Socio determina si cuenta con el código de descuento y Venta decide cual es el precio de venta final, habiendo calculado ya IVA, descuentos y total de la venta.

A quien delega: La clase GestorVentas delega a Artículo la confirmación de su stock, delega a Socio si hay código de descuento o no y delega a Venta que haga el cálculo final del valor de la venta.

Diagrama de secuencia:

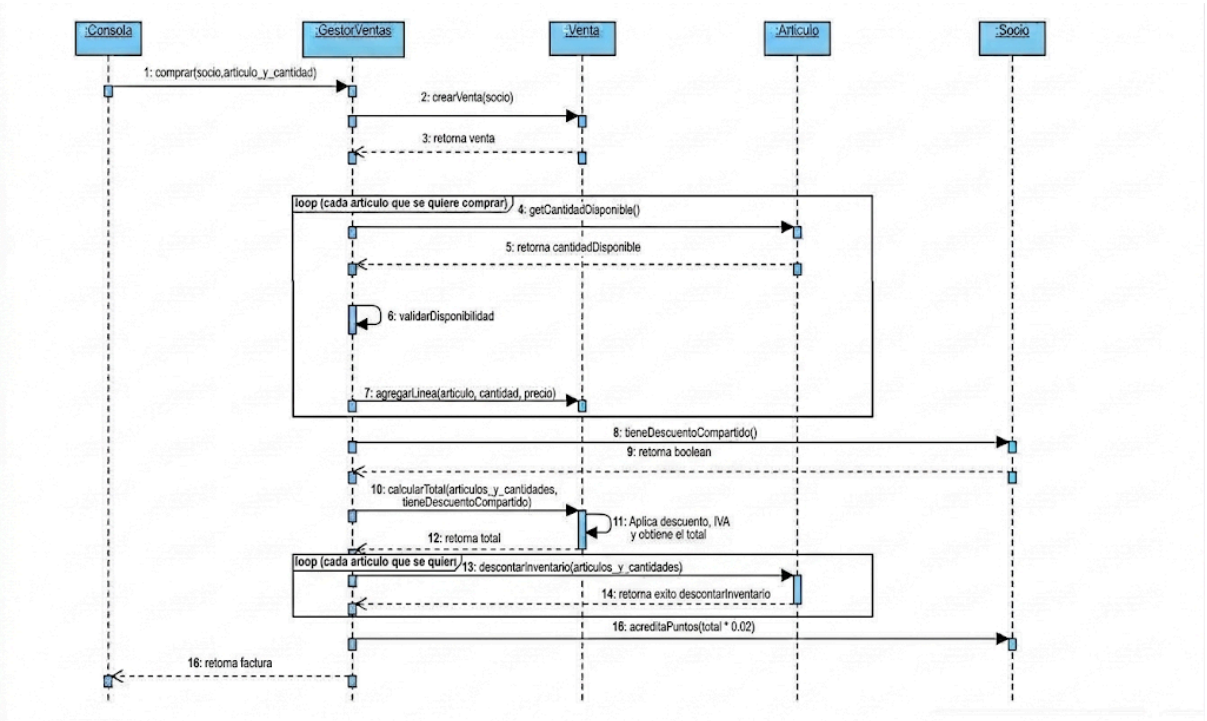


Figura 6. Diagrama de secuencia (Comprar en tienda).

6. Control

6.1. Estilo de control

De las anteriores funcionalidades críticas se detectaron los siguientes estilos de control:

-Verificar rol de usuario (a): El estilo de control de esta funcionalidad es de tipo DISPERSO. Esto, ya que no hay una sola clase central que verifique los roles, sino, que cada subclase de usuario responde mediante métodos puedeX() los permisos que tiene asignados y cada método que se realice verifica que estos métodos puedeX() sean verdaderos para así poder ejecutarse. Es decir que sigue el estilo de dispersión, donde las decisiones se reparten entre

muchos objetos pequeños, como en el anterior caso donde las decisiones están repartidas entre los métodos del sistema y las subclases de usuario.

-Inscribir equipo (b): El estilo de control de esta funcionalidad es de tipo DELEGADO. En este caso el equipo administra y reparte tareas a competencia e InscripcionCompetencia, es decir que las clases colaboran entre sí para llegar a un resultado. Es decir que la clase GestorEquipos se establece como el coordinador que delega tareas a otras clases (en este caso a la clase equipo que hace le hace preguntas a la clase Competencia y órdenes de creación a la clase InscripcionCompetencia), cumpliendo así con los fundamentos que sigue el estilo de control tipo delegado.

-Alinear jugadores a un partido(c): El estilo de control de esta funcionalidad es de tipo DELEGADO. En este caso el equipo administra y reparte tareas a las clases DeporteConjunto y Socio, es decir que las clases colaboran entre sí para llegar a un resultado. Significa que la clase GestorEquipos se establece como el coordinador que delega tareas a otras clases (en este caso le solicita al Equipo que se comunique con DeporteConjunto para saber el mínimo de jugadores habilitados y le solicita a Socio el estado de cada socio), cumpliendo así con los fundamentos que sigue el estilo de control tipo delegado.

-Comprar en tienda(d): El estilo de control de esta funcionalidad es de tipo DELEGADO. En este caso GestorVentas administra y reparte tareas a las clases Artículo, Socio y Venta, es decir que todas las clases colaboran para que se pueda llegar a un resultado. Significa que la clase GestorVentas funciona como el coordinador que delega tareas a las demás clases (Artículo confirma stock, Venta calcula total de compra y Socio determina si hay descuento o no). Esto significa que cumple con los fundamentos que sigue el estilo de control tipo delegado.

7. Alto Nivel

7.1 Clases y relaciones



Figura 7. Diagrama de Alto nivel (UML)

Link para acceder al diagrama:
https://drive.google.com/file/d/1bn7P6vCOP9SEYXf_2-521mN7y4LqbWzi/view?usp=sharing
(Importante abrirlo en Draw.io)

8. Detallado

8.1 Atributos, métodos y cardinalidad.

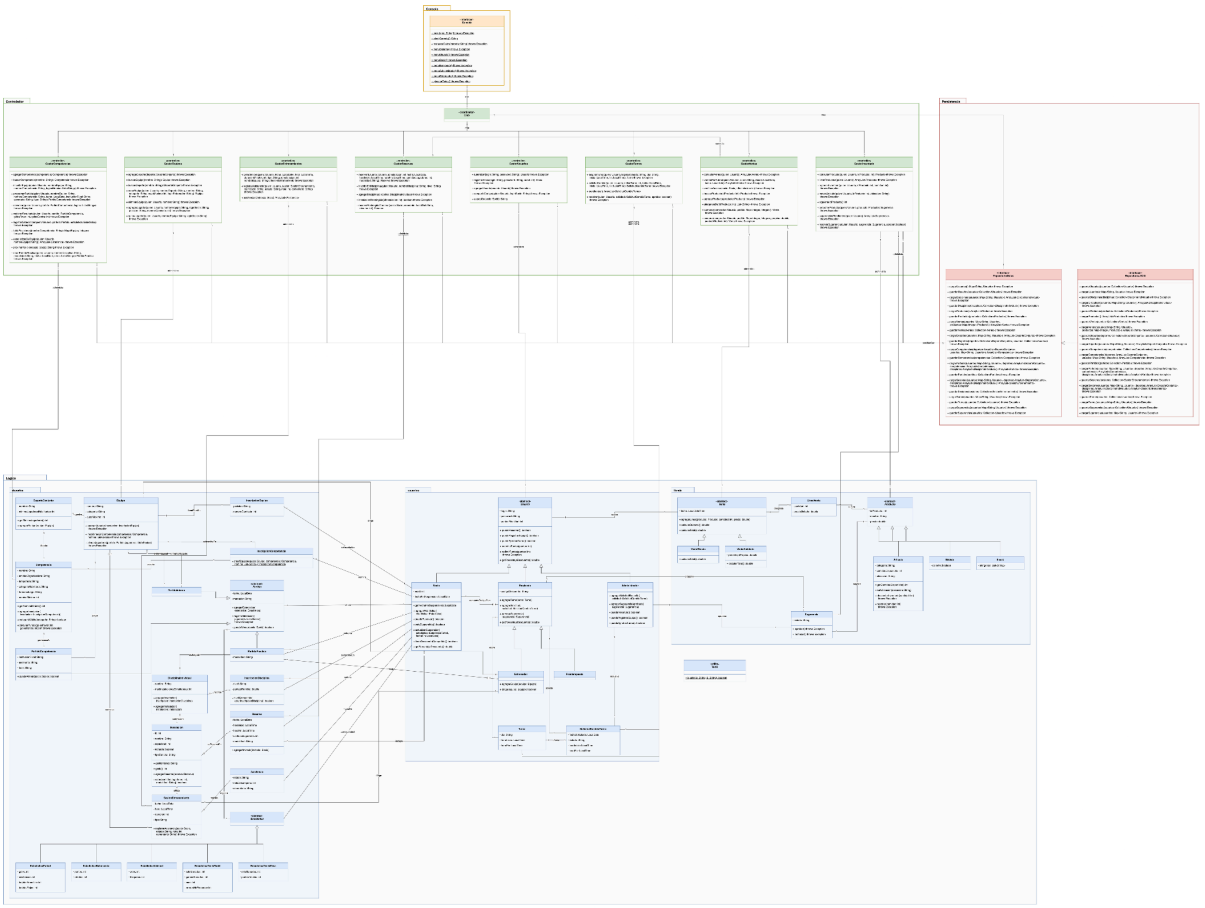


Figura 8. Diagrama detallado (UML)

Link para acceder al diagrama:

https://drive.google.com/file/d/1F_zht0dtH2yL3Vzl-pZG3YhGFAomKPPP/view?usp=sharing(Importante abrirlo en Draw.io)

Nota: Los métodos que rechazan una operación lanzan una excepción con el motivo de RNF2. Pero no se dibujaron en el diagrama para no sobrecargarlo.

8.2 Descripción de cada clase

Paquete consola

Clase	Descripción

Consola	Esta clase se encarga de gestionar todo el sistema, esta se relaciona directamente con club, y de consecuencia se relaciona con todos los paquetes, permitiendo realizar todos los métodos establecidos. Esta consola funciona con base en los gestores. Si un gestor determina un usuario y ese tipo de usuario no puede acceder a un método, la consola debe restringe el acceso a estos métodos.

Paquete usuarios

Clase	Descripción
Usuario	Se encarga de almacenar la información de cada persona, la cuál posee su propio login, password y una determinada cantidad de puntos de fidelidad. Define métodos abstractos que cada subclase debe adaptar así misma, estos métodos responden cuales son los permisos de cada usuario dentro del sistema (puedeX()).
Socio	Se encarga de almacenar la información de cada usuario que sea socio del club. Guarda la edad, que inscribe, sus reservas y sus compras. Cumple con los métodos de su clase padre Usuario, donde asigna los valores a los métodos puedeX() según los permisos otorgados dentro el sistema.
Empleado	Se encarga de almacenar la información de cada usuario que sea empleado del club. Almacena turnos y código de descuento, como atributos abstractos. Esto, con el fin que cada clase hija almacene esta información por igual. Puede compartir su código de descuento con la clase Socio.
Administrador	Se encarga de almacenar la información de cada usuario que sea administrador del club. Esta misma determina los permisos que tiene el administrador para gestionar el

	sistema, donde puede resolver solicitudes de cambio de turno, entre otras acciones más.
Entrenador	Se encarga de almacenar la información de cada empleado que sea entrenador. El objetivo de esta clase es dirigir equipos y sesiones de entrenamiento, por lo tanto, esta relacionada con clases como SesionEntrenamiento, Equipo o PartidoPractica.
Fisioterapeuta	Se encarga de almacenar la información de cada empleado que sea fisioterapeuta. El objetivo de esta clase es almacenar los fisioterapeutas en turno, con tal de que para gestionar los cambios de turno se cumpla con las restricciones establecidas.
Turno	Se encarga de almacenar la información de un turno de trabajo. En esta se guarda el día en que se ejecuta el turno, la hora a la que inicia este y finalmente la hora a la que termina. Se encarga de determinar el turno que debe cumplir cada trabajador y en caso de que se cumpla una solicitud de cambio de turno, se modifica.
SolicitudCambioTurno	Guarda las peticiones de cambio de turno, junto a su fecha, hora inicio y hora fin solicitada más el estado. En caso de ser rechazada no altera la clase turno, en caso de ser aceptada, altera el turno.

Paquete Deportes

Clase	Descripción
DeporteConjunto	Esta clase se encarga de almacenar el tipo de deporte en conjunto (Fútbol, baloncesto o voleibol). En este caso almacena el nombre del deporte junto a los jugadores mínimos habilitados que se necesitan para poder jugar este mismo.
Equipo	Esta clase se encarga de agrupar a los jugadores de un deporte en específico en un mismo equipo. Cada equipo viene definido por un nombre, una categoría y un cupo

	máximo de jugadores por equipo.
InscripcionEquipo	Esta clase se encarga de almacenar las inscripciones que realice un determinado socio para inscribirse a un equipo. Dentro de esta clase se almacena la posición y número de camiseta de socio que se está inscribiendo. Se relaciona con la clase equipo para poder almacenar la inscripción.
Competencia	Esta clase se encarga de almacenar toda la información sobre las competencias que se realicen.Incluye el nombre de la competencia, la entidad que la organiza, la temporada que se juega, la categoría permitida para jugar, el formato de juego que se va a usar y finalmente la nómina mínima de jugadores a tener en cuenta durante la competencia. Es gestionada por el GestorCompetencia y se relaciona directamente con las clases que permiten la ejecución de una competencia.
InscripcionCompetencia	Esta clase se encarga de almacenar la inscripción de un equipo a una competencia. Vincula directamente un equipo a una competencia en específico.
Partido	Esta clase padre almacena toda la información de los partidos que se realicen. Registra fecha y marcador de un partido junto a sus estadísticas (permitido gracias a la relación que tiene con la clase estadística). Posee atributos abstractos para sus clases hijas que son fecha y marcador.
PartidoInterno	Esta clase es subclase de la clase padre partido. Cumple con los atributos abstractos de su padre que son fecha y marchador y a su vez permite un partido entre dos equipos del club.
PartidoCompetencia	Esta clase es subclase de la clase padre partido. Cumple con los atributos abstractos de su padre que son fecha y marchador y a su vez permite un partido entre dos equipos rivales. Almacena también la institución contra la que se juega, el escenario en que se juega y la fase de la competencia en que se realiza.

PartidoPractica	Esta clase es subclase de la clase padre partido. Cumple con los atributos abstractos de su padre que son fecha y marchador y a su vez permite un partido entre dos equipos que buscan practicar. Almacena también la modalidad de juego del partido.
DisciplinaIndividual	Esta clase almacena todos los deportes que se cataloguen como una disciplina individual (Tenis, padel o tenis de mesa). Guarda el nombre de la disciplina y a su vez el máximo de instalaciones simultáneas que puede usar cada deporte en el mismo tiempo.
InscripcionDisciplina	Esta clase se encarga de almacenar las inscripciones que realice un determinado socio para inscribirse a un deporte individual. Dentro de esta clase se almacena el nivel y el puntaje de ranking de cada socio que se está inscribiendo. Se relaciona con la clase disciplinaindividual para poder almacenar la inscripción.
Instalación	Almacena las instalaciones disponibles dentro del club deportivo. En esta se guarda el tipo de instalación (varía según el deporte que se vaya a jugar dentro de esta). Se relaciona directamente con Disciplinas, Reservas y sesiones de entrenamiento.
Reserva	Guarda la información de las reservas que se realicen dentro del club deportivo. En esta se almacena la fecha de la reserva, la hora de inicio y fin de la reserva, la cantidad de socios que van a participar a la reserva y la modalidad de esta misma. Se relaciona directamente con el gestor de reservas que es el que permite la realización de estas.
Estadistica	Almacena las estadísticas de un jugador en específico. Es una clase padre que no posee atributos abstractos, ya que cada clase hija mide estadísticas según el funcionamiento del deporte que esté midiendo paralelamente.
EstadisticaFutbol	Clase hija de la clase Estadistica. Esta clase se encarga de almacenar toda las estadísticas de un jugador en específico durante un

	partido de fútbol en específico. Esta almacena: los goles realizados, las asistencias realizadas, las tarjetas amarillas obtenidas y las tarjetas rojas obtenidas.
EstadisticaTenisMesa	Clase hija de la clase Estadistica. Esta clase se encarga de almacenar todas las estadísticas de un jugador en específico durante una partida de tenis de mesa en específico. Esta almacena: los sets ganados y los puntos totales realizados.
EstadisticaBaloncesto	Clase hija de la clase Estadistica. Esta clase se encarga de almacenar todas las estadísticas de un jugador en específico durante un partido de baloncesto en específico. Esta almacena: los puntos y los rebotes de cada jugador.
EstadisticaVoleibol	Clase hija de la clase Estadistica. Esta clase se encarga de almacenar todas las estadísticas de un jugador en específico durante un partido de voleibol en específico. Esta almacena: los aces realizados y los bloqueos realizados durante el partido.
EstadisticaTenisPadel	Clase hija de la clase Estadistica. Esta clase se encarga de almacenar todas las estadísticas de un jugador en específico durante un partida de padel en específico. Esta almacena: los sets ganados, los juegos ganados y los aces realizados.
SesionEntrenamiento	Esta almacena la sesión de entrenamiento que se va a realizar según un entrenador y los diversos socios que vayan a participar en esta. Con esto se guarda la fecha en que se realizó, la hora a la que se realizó, la duración que tuvo y finalmente el tipo de entrenamiento que fue.
Asistencia	Esta clase almacena las asistencias de cada socio durante una sesión de entrenamiento. Esta guarda el estado, la nota de desempeño que tuvo el socio durante la sesión y un comentario opcional que puede añadir el entrenador respecto a la sesión.

Paquete Tienda.

Clase	Descripción
Producto	Se encarga de almacenar toda la información sobre cada producto. Esto lo hace a través de dos atributos abstractos: nombre y precio. Funciona como clase padre y además puede ser modificada por sugerencias y se tiene en cuenta en las líneas de venta a la hora de generar la factura de una venta.
Articulo	Esta clase se encarga de almacenar la información de la subclase Artículo. Toma los atributos abstractos de la clase producto y los modifica según esta misma. Además incluye atributos propios como lo son: categoría, cantidad disponible y ubicación.
Bebida	Esta clase se encarga de almacenar la información de la subclase Bebida. Toma los atributos abstractos de la clase producto y los modifica según esta misma. Además incluye un atributo que permite determinar si la bebida es fría o caliente.
Snack	Esta clase se encarga de almacenar la información de la subclase Snack. Toma los atributos abstractos de la clase producto y los modifica según esta misma. Además almacena los alérgenos que componen el snack.
Venta	Esta clase almacena la información de una venta y se encarga de realizarla. Dentro esta se almacena la fecha en la que se realizó la compra mediante un atributo abstracto. Su objetivo es agrupar las líneas de compra formando una factura y calcula el total de la compra para así poder realizarla.
VentaTienda	Almacena todas las ventas que tengan que ver con tienda, toma en cuenta el atributo abstracto de su clase padre venta y lo modifica según cuando se efectúe una venta en tienda. Toma en cuenta el impuesto para el cálculo del total de venta.
VentaCafeteria	Almacena todas las ventas que tengan que ver con cafetería, toma en cuenta el atributo

	abstracto de su clase padre venta y lo modifica según cuando se efectúe una venta en tienda. A su vez para el total de la venta tiene en cuenta una propina y a su vez el impuesto.
LineaVenta	Almacena cada renglón de una transacción. En esta se tiene en cuenta la cantidad de veces que se compró determinado producto y el precio unitario de este. Sirve para armar la venta y calcular el total de esta misma.
Sugerencia	Almacena la información de productos que proponga un determinado empleado. Aca se incluye el estado de la solicitud, el cual solo puede ser cambiado por el administrador. En caso de ser aceptado se creará un nuevo objeto en la clase producto, en caso de ser rechazado, todo se mantendrá igual y la sugerencia se mantendrá en estado de rechazado.

Paquete controlador.

Clase	Descripción
GestorUsuarios	Esta clase se encarga de verificar el login y entregar el perfil de cada usuario según su tipo. Interactúa directamente con la clase usuarios para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorEquipos	Esta clase se encarga de crear nuevos equipos y valida la entrada de cada jugador a un determinado equipo. Interactúa directamente con la clase equipo para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorCompetencias	Esta clase se encarga de validar las inscripciones de equipos, los jugadores convocados y suspendidos en una competencia. Interactúa directamente con la clase competencia para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.

GestorReservas	Esta clase se encarga de validar y registrar todas las reservas y partidos de práctica que se deseen realizar. Interactúa directamente con la clase reserva para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorEntrenamientos	Esta clase se encarga de programar sesiones y registrar asistencia y evaluación de cada sesión realizada. Interactúa directamente con la clase sesionentrenamiento para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorVentas	Esta clase se encarga de validar las ventas registradas y de registrar nuevas ventas. Interactúa directamente con la clase venta para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorInventario	Esta clase se encarga de controlar la existencia de productos como también sugerencias de nuevos productos. Interactúa directamente con la clase producto para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
GestorTurnos	Esta clase decide los cambios de turno según el mínimo de personal y la decisión que tome el administrador. Interactúa directamente con la clase turno para realizar todas las gestiones que sean necesarias durante la ejecución del sistema.
RepositorioDatos (interfaz)	Define como la lógica pide guardar y cargar datos
RepositorioJSON	Implementa el repositorio con archivos JSON
Consola	Ejecutan los casos de éxito y rechazo de los RF
Club	Esta clase se encarga de crear los gestores y el repositorio y los conecta entre sí. Se relaciona directamente con todos los gestores existentes.

Paquete Persistencia

Clase	Descripción
RepositorioDatos (interfaz)	Esta clase se encarga de definir la lógica de cómo se guardaran y cargaran los datos dentro del sistema. Se relaciona directamente con la clase club y los gestores.
RepositorioJSON	Esta clase se encarga de implementar el repositorio con archivos de tipo JSON y a su vez se relaciona directamente con la clase reposiotoriodatos para almacenar toda la información que sea necesaria durante la ejecución del sistema.

9. Decision

9.1 Sección 1. Contexto

Decisión 1.A. El Ranking no se recalcula.

- Problema: el enunciado menciona que existe un puntaje de ranking, sin embargo no se define quién registra el resultado de un partido de práctica.
- Alternativas: la primera alternativa era que después de cada partido se recalculara el puntaje. La otra era conservarlo sin recalcular.
- Elección: conservarlo como atributo de InscripcionDisciplina, sin recalcular.
- Pros: una implementación más simple sin sobre complicar reglas no definidas.
- Cons: El puntaje queda estático y no refleja el desempeño en los partidos.

Decisión 1.B. Bebida caliente: validar solo al vender y al reservar.

- Problema: el enunciado prohíbe entrar con bebida caliente a una cancha de pádel o de tenis de mesa techada, sin embargo el sistema no tiene como saber que lleva una persona consigo.
- Alternativas: control físico de acceso (imposible modelar), o validar en momentos en los que el sistema sí tiene cómo saber.
- Elección: rechazar la venta si el socio tiene una reserva vigente, y rechazar la reserva si compró una bebida caliente en los últimos 30 minutos.
- Pros: la regla queda aplicada en el sistema, sin depender de la consola.

- Cons: el sistema no impide que alguien consiga una bebida caliente por otro medio.

Decisión 1.C. Contraseñas sin cifrar

- Problema: todos los usuarios tienen login y contraseña, pero no se especifica si se necesita seguridad avanzada.
- Alternativas: guardar con hash, o guardar tal cual.
- Elección: Guardarlas tal cual en la carpeta de datos, aprovechando que el enunciado dice que solo la aplicación lee y escribe esos archivos.
- Pros: facilidad al implementar sin cifrado ni recuperación.
- Cons: si esa afirmación del enunciado no existiera, tocaria rehacer como se guarda la contraseña.

Cambios frente al análisis.

Creamos un bloque de RF para el sistema (guardar, recuperar y que el socio solo vea lo suyo). Se ajustaron los RF para reflejar las decisiones del 1.A y 1.B, también se arreglaron las tarifas del caso (IVA en tienda, impuesto al consumo y propina solo en cafetería y los puntos de fidelidad del 2% en toda venta). También quitamos los RF de Fisioterapeuta ya que hereda de Empleado y no tiene RF propios. Adicional se agrega un RF, el cual verifica que se puedan registrar los resultados de la competencia (el Entrenador lo hace). Las competencias no se crean desde el sistema, sino que se cargan como datos iniciales del club.

9.2 Sección 2. Componentes

Decisión 2.A. Tres componentes.

- Problema: separar las reglas del club de la consola y de los archivos.
- Alternativas: todo en un solo bloque, o dividirlo por Consola, Lógica y Persistencia.
- Elección: tres componentes, siguiendo el modelo MVC propuesto por el monitor. La interfaz del repositorio vive en la lógica y la implementa la persistencia.
- Pros: la lógica no conoce la consola ni el formato de los archivos, y cada cambio toca un solo paquete.
- Cons: más paquetes e interfaces para una aplicación pequeña.

9.3 Sección 3. Roles

Decisión 3.A. Gestores como controladores

- Problema: no se sabía quién recibe las peticiones de la consola y coordina las reglas que cruzan varios objetos.

- Alternativas: las clases del dominio reciben directo, o un solo controlador central, o un gestor por área.
- Elección: un gestor por área. Club solo los crea y conecta.
- Pros: cada decisión que cruza objetos, como un jugador por equipo o la bebida caliente, tiene dueño claro.
- Cons: más clases, y algunos gestores quedan más cargados.

9.4 Sección 4. Responsabilidades

Decisión 4.A: Excepciones con Bool

- Problema: El RNF2 requiere un aviso que explique cualquier falla del sistema.
- Alternativas: Retorno Boolean, o un sistema de excepción.
- Elección: Sistema de excepción.
- Pros: Cuando un proceso falla, el sistema retorna un mensaje detallando exactamente el problema.
- Cons: Al manejar casos de una falla singular el costo de computación aumenta y en casos donde se encuentren varias fallas puede que la excepción solo retorne el razonamiento de un solo error.

Decisión 4.B: La suspensión la decide Socio

- Problema: quién decide si un jugador está suspendido. (R9)
- Alternativas: GestorCompetencias decide, o una clase Sanción, o Socio decide con las tarjetas que guarda su estadística.
- Elección: EstadisticaFutbol guarda las tarjetas, y Socio.estaSuspendido() suma las de la competencia y aplica la regla.
- Pros: Coincide con secuencia donde Equipo ya le pregunta a Socio, y el gestor solo registra estadísticas.
- Cons: Socio depende de las estadísticas de cada competencia.

Decisión 4.C: Manejo de tildes y mayúsculas

- Problema: El sistema considera que la misma palabra, con o sin tilde o mayúsculas, son palabras diferentes.
- Alternativas: Guardar una lista de palabras normalizadas, repetir la normalización en cada comparación, o crear una clase suplementaria cual normaliza el texto.
- Elección: La nueva clase Texto se llama para normalizar todos los datos de entrada.

- Pros: Como la misma regla se aplica en la mayoría de comparaciones los casos donde el sistema no reconozca los datos de entrada se solucionan.
- Cons: El proceso de normalización se aplica a todos los datos de entrada lo cual aumenta la complejidad del programa y el sistema depende de la clase auxiliar sin modo propio de procesar datos que no cumplen con las condiciones esperadas.

Decisión 4.D: Acceso a métodos

- Problema: Los accesos a métodos deben ser definidos por subclase.
- Alternativas: Usar una comparación “if” para verificar el acceso, métodos se implementan por separado en cada rol, o solo implementar métodos que se usan en casos específicos de cada rol y permisos individuales.
- Elección: Los permisos de Usuario quedan en False, el socio puede reservar mientras que el administrador puede inscribir equipos y aprobar turnos. El entrenador no tiene permiso directo: GestorEquipos usa Entrenador.dirige(equipo)
- Pros: No se hacen comparaciones para verificar el acceso y todos los roles quedan con la funcionalidad esperada.
- Cons: el control del entrenador para inscribir equipos queda fuera del método local.

Decisión 4.E: Inscripción a partidos

- Problema: La inscripción, el entrenamiento, los partidos y la registración a una competencia se reparten entre las clases involucradas.
- Alternativas: cada clase decide lo que conoce y equipo guarda las respuestas, el gestor valida todo, o implementar métodos polimórficos.
- Elección:agregarJugador revisa cupo, duplicados, camisetas únicas y la edad. inscribirseEnCompetencia revisa la categoría, la nómina mínima y que los jugadores sean del equipo.
- Pros: Cada clase decide lo que conoce por sí mismo.
- Cons: Como los jugadores se comparan con == puede causar problemas si la persistencia carga a un socio varias veces.

Decisión 4.F: Implementación de sistema de impuestos

- Problema: se tiene que aplicar los impuestos, la propina y los descuentos en los casos necesarios de venta para la tienda.
- Alternativas: usar un mapa para calcular, o que las líneas se sumen directamente.

- Elección: Venta suma sus LineaVenta y aplica comprador.getPorcentajeDescuento(). VentaTienda y VentaCafeteria suman sus impuestos sobre super.calcularTotal(). El porcentaje de propina se puede modificar.
- Pros: Sirve para todos los productos disponibles y el total coincide con las lineas.
- Cons: Las líneas deben agregarse antes de calcular.

9.5-6 Sección 5-6. Secuencias y control:

Decisiòn 5-6.A

- Problema: Elección estilo de control para la secuencia 1 de validar usuario.
- Alternativas: Se tenían las 3 opciones de estilos de control: delegado, disperso o centralizado.
- Elección: Se eligió utilizar el estilo de control disperso. Utilizar el estilo delegado o centralizado era poco eficiente ya que esta secuencia funciona con una gran cantidad de clases, por lo tanto, mantener un orden lógico entre estas era poco eficiente y poco coherente.
- Pros: Modifica cada subclase según las características necesarias sin necesidad de perjudicar el resto de clases existentes. Cada clase contiene su propia información y no requiere ser evaluada por funciones nativas o auxiliares.
- Cons: Cada clase tiene su propia información y no sigue un orden específico, esto dificulta la lectura y entendimiento en caso de no poseer el contexto.

Decisiòn 5-6.B

- Problema: Elección estilo de control para la secuencia 2 de inscribir equipo, 3 de alinear jugadores y de 4 de comprar tienda.
- Alternativas: Se tenían las 3 opciones de estilos de control: delegado, disperso o centralizado.
- Elección: Se eligió utilizar el estilo de control delegado. Se elige este estilo ya que todas las validaciones dependen de las demás clases, es decir, el resultado esperado de una consulta puede depender de un antecesor. Esta misma razón genera que sea necesario usar el método delegado, para así mantener una fuerte cohesión entre las decisiones que se tomen para generar un resultado.

- Pros: Fácil de identificar la conexión entre clases, estructura ordenada y en caso de que falle una decisión en especifico es mas fácil ubicar los errores.
- Cons: Mayor dificultad de actualización en caso de que sea necesario. Al ser estilo delegado siempre que se cambie una clase se debe verificar que el resto quede funcionando de manera correcta y consistente a la modificada.

Decisiòn 5-6.C

- Problema: En que momento se debería descontar el inventario al momento de una compra?
- Alternativas: Se tenían 2 opciones, descontar al momento de agregar la línea a la factura o al momento de finalizar la compra.
- Elección: Se eligió eliminar el inventario después de finalizar la compra. Esto se debe al RFN1, ya que este dice que en caso de no finalizar una compra debe mantenerse igual el inventario presente. Por lo tanto, se eligió eliminar el inventario después de terminar cada compra.
- Pros: Evita falsas estimaciones del inventario disponible, pues evita descontar inventario cuando ni siquiera fue adquirido. Permite llevar un control exacto de cuando inventario hay presente.
- Cons: En caso de que falle la suma de puntos y no se realice la compra se descontará inventario a pesar de que la compra haya fallado.

10. Documentación IAG.

Samuel Z.

Herramienta: Claude (Anthropic, 2026)

<u>Documento de diseño.</u>

Resumen: el uso fue de guía y retroalimentación. El contenido venía de la Entrega 1 y de nuestro trabajo previo, y lo redactamos nosotros. Se consultó a Claude para discutir y para pedir ejemplos.

Requerimientos funcionales y no funcionales.
Prompt: Se le pasaron los RF's del documento de análisis. "estos son los requerimientos de la versión anterior; ayúdame a debatir cuáles quitar y cuáles agregar."
Resultado: Tuvimos un flujo de conversación en el cual señalo RF's repetidos, y algunos que eran muy complejos.
Decision: Decidimos que RF se reducían, cuáles se agregaban y cuáles pasaban a

Para el resto de incisos del 1-4. Solo se le pasó el documento de análisis, y se le pidió que nos ayudara con algunos ejemplos en las tablas, como las responsabilidades, los componentes y los roles. Además de esto no se utilizó mucho más.

Implementación Java
Para la implementación, si se utilizo mucho más la IAG, ya que yo implemente Persistencia y los Controladores, los cuales eran complejos. La mayoria del uso fue guia para el codigo, que lo revisara, y que lo probara con pruebas.
GestorEntrenamientos.
Prompt: Se le paso el UML, se le explico el flujo del documento de diseño. Le pedí que me explicara como implementar las funciones de los metodos de el gestor, que logica tenia que importar y que clases estaban conectadas a este para que el flujo pasara y las pruebas tambien.
Respuesta:
GestorEntrenamientos, paso a paso

1. Atributos y constructor. Guarda repositorio, usuarios, gestorEquipos, gestorReservas y la lista sesiones. En el constructor, la lista se llena con repositorio.cargarSesiones(...). Por eso el constructor lleva throws Exception.

2. getSesiones(). Solo devuelve la lista.

3. crearSesion (RF1 de Entrenador). Es el método más importante. Valida en este orden y lanza new Exception("motivo") en cada fallo:

quien instanceof Entrenador. Si no, rechazas.
Que el entrenador dirija ese equipo. Busca el equipo con gestorEquipos.buscarEquipo(nombre) y revisa si el entrenador lo dirige.
Duración positiva y fecha no pasada.
Instalación libre: recorre instalacion.getReservas() y luego las otras sesiones. La condición de traslape es inicioA < finB && finA > inicioB.

Solo después de pasar todas las validaciones creas el SesionEntrenamiento, haces setEquipo, lo agregas a la lista y llamas repositorio.guardarSesiones(sesiones). Primero validar, después modificar: así una operación rechazada no deja nada a medias (tu RNF de consistencia).

4. registrarAsistencia (RF2 de Entrenador). Valida que quien sea el entrenador de la sesión, que el login exista y sea un Socio (usa instanceof), y que ese socio esté en la plantilla del equipo de la sesión. Luego llama sesion.registrarAsistencia(...), que es de Tulio, y guardas.

5. asistenciasDe(Socio) (RF12 de Socio). Dos for anidados: por cada sesión, por cada asistencia, si a.getSocio() == socio la agregas a una lista nueva.

Trampas que me pasaron a mí
Se te olvidará guardarSesiones en algún método. Si la prueba de persistencia falla, empieza por ahí.
Para comparar objetos usa == solo si son el mismo objeto del mapa. Para textos usa .equals.
Los mensajes de rechazo sirven como RNF2: dicen el motivo, no un genérico.
Cómo seguimos

Escribe GestorEntrenamientos y pásame lo que te dé error o lo que dudes. Te corrijo sin reescribirlo. Con esa base haces el resto.

Decisión: Se iba escribiendo el código poco a poco y se iba consultando con la IAG, en algunos puntos de no retorno se le pedía pseudocodigo. (Esto sucedió más en persistencia)

Pero al final cuando ya se tenía la solución, se le mando el código completo y le pedimos que corriera el código con la lógica y pruebas para ver si pasaba.

GestorCompetencias. (Era el más grande)
Prompt: Como para el anterior gestor se le mostro todos los RF's, el UML y pues como queriamos el flujo. Se le paso el enunciado, y se le pidio ayuda escribiendo este codigo ya que estaba muy largo. Le pedimos una plantilla y una lista de instrucciones.
Respuesta:
Antes de escribir: qué ya hace Tulio

No reimplementes estas cosas, llámalas:

competencia.calcularPuntos(golesFavor, golesContra) calcula los puntos y rechaza resultados inválidos, como un empate en baloncesto.
equipo.inscribirseEnCompetencia(competencia, nomina) hace la inscripción.
equipo.alinearJugadores(partido, jugadores) valida y arma la alineación.
socio.actualizarSuspension(estadistica, fechaFin) aplica la regla de tarjetas.
inscripcionA.nivelCompatible(inscripcionB) compara niveles.

Tu trabajo en el gestor es: buscar los objetos, validar quién puede hacerlo, llamar a la lógica y guardar.

Pasos

1. Atributos y constructor.
Guarda repositorio, usuarios, gestorEquipos, gestorReservas, y dos listas: competencias y partidos. En el constructor las llenas con repositorio.cargarCompetencias(...) y luego cargarPartidos(...). Ese orden importa: los partidos necesitan las competencias ya cargadas. Agrega public static final int DIAS_SUSPENSION = 7;.

2. Getters y agregarCompetencia.
Los getters devuelven las listas. agregarCompetencia es un dato inicial: rechaza si ya existe una con ese nombre, agrega y guarda.

3. buscarCompetencia(String nombre).
Recorre la lista comparando con Texto.iguales. Si no la encuentra, throw new Exception("No existe la competencia ..."). Úsalo en casi todos los métodos de abajo.

4. Tres métodos de ayuda privados. Escríbelos ya para no repetir código:

exigirEntrenadorDe(Usuario quien, Equipo equipo): lanza excepción si quien no es Entrenador o no dirige ese equipo.
buscarSocio(String login): busca en el mapa usuarios y valida con instanceof Socio.
estaInscrito(Equipo, Competencia): recorre competencia.getInscripciones() y compara con ==.

5. inscribirEquipo (RF4 de Entrenador). Orden de las validaciones:

Busca el equipo con gestorEquipos.buscarEquipo.
exigirEntrenadorDe.
Busca la competencia.
El deporte de la competencia debe ser el del equipo.
Que el equipo no esté ya inscrito.
Convierte la lista de logins en una lista de Socio con buscarSocio.

Solo entonces llamas a equipo.inscribirseEnCompetencia(...) y guardas con guardarCompetencias.

6. programarPartido.
Mismas dos primeras validaciones, pero además que el equipo esté inscrito (estaInscrito). Crea el PartidoCompetencia, agrégalo a partidos, guarda con guardarPartidos y devuélvelo.

7. convocar (RF3 de Entrenador).
exigirEntrenadorDe(quien, partido.getEquipo()), convierte los logins en socios y llama alinearJugadores. Guarda partidos.

8. registrarResultado.

Valida entrenador. Llama a calcularPuntos solo para que rechace los resultados inválidos. Luego partido.setMarcador(golesFavor + "-" + golesContra) y guarda.

9. registrarEstadistica. Es el método más delicado:

Valida quién puede, según el tipo de partido. Usa instanceof para distinguir PartidoCompetencia, PartidoInterno y PartidoPractica.
Verifica que el socio de la estadística haya participado (está en getAlineados(), o en getJugadores() si es de práctica).
Agrega la estadística al partido y al socio.
Si la estadística es de fútbol (instanceof EstadisticaFutbol), llama a actualizarSuspension con la fecha del partido más DIAS_SUSPENSION, y guarda también los usuarios, porque la suspensión vive en el socio.
Guarda partidos.

10. tablaPosiciones.

Arma un mapa Equipo → puntos con todos los inscritos en 0 (usa LinkedHashMap para conservar el orden).
Recorre los partidos: los que sean de esa competencia y tengan marcador, separa el marcador con split("-"), convierte con Integer.parseInt y suma el resultado de calcularPuntos.
Ordena de mayor a menor. Esa es la parte nueva de verdad: copia las entradas a una lista y ordénala con un Comparator. Si no lo conoces, dime y lo vemos aparte.

11. estadisticasDeEquipo (RF5 de Entrenador).
Valida entrenador. Recorre equipo.getPlantilla() y junta getSocio().getEstadisticas() en una lista nueva.

12. proximaPosicion (RF1 de Socio).
Busca el partido de competencia más próximo, sin marcador y con fecha de hoy o posterior, donde el socio esté alineado. Si no hay, lanza excepción. Si hay, busca al socio en la plantilla del equipo y devuelve su getPosicion().

13. crearPartidoPractica (RF3 de Socio). Valida en este orden:

quien es Socio.
Fecha no pasada.
La disciplina existe (la buscas en gestorReservas.getDisciplinas()).
Modalidad: Sencillos pide 1 más, Dobles pide 3 más.
El creador está inscrito en la disciplina.
Para cada otro login: si es Entrenador, es el rival, que queda exento de nivel. Si es Socio, debe estar inscrito, con nivel compatible y no repetido.

Luego creas el PartidoPractica, le pones el rival con setRival, lo agregas y guardas.

Decision: Este fue la respuesta base de la cual se comenzó a implementar el gestor, haciendo verificaciones a mitad de camino, pidiendo ayuda y retroalimentación, y finalmente pidiendo que corriera pruebas unitarias o de validación.

Persistencia

La persistencia fue lo que mas tiempo me tomo, revise la presentación y se me complico mucho. Le pedi a Claude que me explicara y no lograba que se me conectaran todos los flujos, que se me guardaran los datos en la carpeta y las demas funcionalidades de este paquete.

Prompt. Fue un prompt muy extenso, le pase el pdf de Persistencia que vimos en clase, el contexto del proyecto, le pedi ayuda con las librerias y que mecanismo ibamos a usar para diseñar la estructura de archivos. Le pase la logica comprimida, y le pase un codigo inicial. El cual estaba casi completamente equivocado y me mostro un arreglo de este.

Respuesta:

El código de las diapositivas es imagen y no pude extraerlo. Lo que sigue usa la API de org.json que la presentación nombra (JSONArray, JSONObject).

Qué tiene que funcionar (tu lista de verificación)
Guardar y cargar de vuelta da los mismos datos (ida y vuelta).
Si el archivo no existe, al cargar devuelve una lista vacía y no falla (primera ejecución).
Una entidad que referencia a otra se guarda con su llave (login, nombre, id) y no con el objeto completo.

Al cargar, primero creas los objetos y después enlazas las referencias.
Un error de lectura o escritura sale como excepción con mensaje claro.
La prueba final reabre Club y comprueba que todo sigue ahí.
Paso 0. Preparación
Crea los paquetes persistencia y la carpeta lib/ si no existe.
Pon json-20240303.jar en lib/.
Clic derecho al proyecto → Build Path → Add JARs → elige ese jar.
Crea la carpeta datos en la raíz del proyecto (clic derecho → New → Folder).
Paso 1. La interfaz RepositorioDatos

Es solo la lista de lo que el sistema necesita, sin implementación. Todos los métodos lanzan Exception. La plantilla:

java
```
package persistencia;

import java.util.*;
import logica.usuarios.Usuario;
// ... importa las demás clases cuando las uses

public interface RepositorioDatos {

    void guardarUsuarios(Collection<Usuario> usuarios) throws Exception;
    Map<String, Usuario> cargarUsuarios() throws Exception;

    void guardarProductos(Collection<Producto> productos) throws Exception;
    ArrayList<Producto> cargarProductos() throws Exception;

    // Las que tienen referencias reciben lo que necesitan para enlazar:
    void guardarVentas(Collection<Venta> ventas) throws Exception;
    ArrayList<Venta> cargarVentas(Map<String, Usuario> usuarios,
                                  Map<Integer, Producto> productos) throws Exception;

    // Repite el patrón: disciplinas, deportes, competencias,
```

```
    // partidos, sesiones, turnos, sugerencias.
}
```

Regla para decidir los parámetros de cargarX: pregúntate "¿a qué otras entidades apunta lo que voy a cargar?". Esas entidades las pasas como parámetro, ya cargadas. Eso te dice el orden en que Club debe cargar: usuarios y productos primero (no apuntan a nada), luego ventas, deportes, competencias, partidos, y así.

Paso 2. RepositorioJSON: esqueleto y dos ayudas

Escribe primero las dos ayudas, porque todos los métodos las usan:

java

```
public class RepositorioJSON implements RepositorioDatos {

    private final File carpeta;

    public RepositorioJSON(String carpetaDatos) {
        this.carpeta = new File(carpetaDatos);
    }

    // Escribe un arreglo JSON en datos/<archivo>
    private void escribir(String archivo, JSONArray arreglo) throws Exception {
        if (!carpeta.exists()) {
            carpeta.mkdirs();
        }
        Files.writeString(new File(carpeta, archivo).toPath(), arreglo.toString(2));
    }

    // Lee datos/<archivo>; si no existe devuelve un arreglo vacío
    private JSONArray leer(String archivo) throws Exception {
        File f = new File(carpeta, archivo);
        if (!f.exists()) {
            return new JSONArray();
```

```
        }
        return new JSONArray(Files.readString(f.toPath()));
    }
```

toString(2) escribe el JSON con sangría de 2 espacios para que sea legible. Si el archivo existe pero está dañado, new JSONArray(...) lanza excepción, y eso ya cubre el punto 5 de la lista.

Paso 3. Un ejemplo completo: usuarios

Hazlo así y verás el patrón de todo lo demás.

Guardar: un objeto JSON por usuario, guardando solo atributos simples y llaves.

java

```
    @Override
    public void guardarUsuarios(Collection<Usuario> usuarios) throws Exception {
        JSONArray arreglo = new JSONArray();
        for (Usuario u : usuarios) {
            JSONObject json = new JSONObject();
            json.put("login", u.getLogin());
            json.put("password", u.getPassword());
            json.put("puntos", u.getPuntosFidelidad());
            if (u instanceof Socio) {
                json.put("tipo", "SOCIO");
                json.put("edad", ((Socio) u).getEdad());
                // si tiene empleado de descuento: guarda SOLO su login
            }
            // else if (u instanceof Administrador) ... "ADMINISTRADOR"
            // else if (u instanceof Entrenador)  ... "ENTRENADOR" + codigoDescuento
            arreglo.put(json);
        }
        escribir("usuarios.json", arreglo);
    }
```

La clave es el campo "tipo": al cargar, te dice qué clase construir.

Cargar, en dos pasadas:

java

```
@Override
public Map<String, Usuario> cargarUsuarios() throws Exception {
    Map<String, Usuario> usuarios = new LinkedHashMap<String, Usuario>();
    JSONArray arreglo = leer("usuarios.json");

    // Pasada 1: crear todos los objetos
    for (int i = 0; i < arreglo.length(); i++) {
        JSONObject json = arreglo.getJSONObject(i);
        Usuario usuario;
        switch (json.getString("tipo")) {
        case "SOCIO":
            usuario = new Socio(json.getString("login"),
                    json.getString("password"), json.getInt("edad"));
            break;
        case "ADMINISTRADOR":
            usuario = new Administrador(json.getString("login"),
                    json.getString("password"));
            break;
        // ENTRENADOR, FISIOTERAPEUTA, EMPLEADO igual
        default:
            throw new Exception("Tipo de usuario desconocido");
        }
        usuario.acreditaPuntos(json.getInt("puntos"));
        usuarios.put(usuario.getLogin(), usuario);
    }

    // Pasada 2: enlazar referencias (ej. empleado de descuento del socio)
    for (int i = 0; i < arreglo.length(); i++) {
```

```
        JSONObject json = arreglo.getJSONObject(i);
        if (json.has("empleadoDescuento")) {
            Socio s = (Socio) usuarios.get(json.getString("login"));
            s.setEmpleadoDescuento(
                    (Empleado) usuarios.get(json.getString("empleadoDescuento")));
        }
    }
    return usuarios;
}
```

Por qué dos pasadas: un socio puede apuntar a un empleado que aparece más abajo en el archivo. En la primera pasada todos existen; en la segunda ya puedes enlazar.

Paso 4. El patrón para el resto de entidades

Para cada entidad repite estos cinco pasos:

Enumera qué atributos tiene la clase de Tulio.
Separa los simples (texto, número, fecha) de los que son referencias a otros objetos.
Guarda los simples tal cual. Las fechas con toString(), y las lees con LocalDate.parse(...), LocalTime.parse(...), LocalDateTime.parse(...).
Las referencias se guardan con la llave: el login del usuario, el nombre del equipo, el id del producto.
Al cargar, busca el objeto por esa llave en el mapa o lista que recibes por parámetro. Si no lo encuentras, lanza excepción con la llave.

Orden recomendado para escribirlas:

Productos (Snack/Bebida: igual que usuarios, con un “tipo”).
Disciplinas (con instalaciones e inscripciones de socios).
Ventas (apuntan a usuario y producto; guardan las líneas dentro).
Deportes con equipos y plantilla (la plantilla guarda login, posición y camiseta).
Competencias (inscripciones con la nómina por logins).
Partidos (3 tipos con “tipo”; las estadísticas van embebidas con su propio “tipo”).

Sesiones y asistencias.
Turnos y solicitudes (viven dentro de cada empleado).
Sugerencias.
Paso 5. Cómo se usa desde los gestores
En el constructor del gestor: lista = repositorio.cargarX(...).
Después de cada cambio: repositorio.guardarX(lista).
En Club, el constructor sin argumentos crea new RepositorioJSON("datos"). Eso responde a la relación «crea» de tu UML.
Paso 6. Probar de a una

No escribas todo y pruebes al final. Después de cada entidad, haz una prueba corta:

java
RepositorioJSON r = new RepositorioJSON("datos_prueba");
r.guardarUsuarios(lista);
Map<String, Usuario> leidos = r.cargarUsuarios();
// compara tamaño y un atributo

Abre el .json generado y míralo: si no lo puedes leer tú, algo está mal.

Errores comunes
Guardar un objeto completo dentro de otro en vez de su llave: el archivo se duplica y se desincroniza.
Olvidar guardar al final de un método del gestor.
json.getString("x") lanza excepción si el campo no existe. Para los opcionales usa json.has("x") antes.
Cargar en el orden equivocado en Club: los parámetros de cargarX te dicen cuál es el orden.

Decisiones: A raíz de estas correcciones se comenzó a hacer la implementación para el RepositorioDatos y el JSON, sin embargo solo pude hacer el caso ejemplo que me mostro, y la mayoría fue casi copiando el código completamente. Entonces le pedí que me enseñara este patrón pero para el resto de Gestores, para disciplinas deportes competencias, etc… Porque si no tenía que copiar y pegar el código.

Persistencia ahora con Disciplinas.

Prompt: Habia varios casos donde el codigo no me funcionaba, el archivo no existía, estaba vacío, estaba dañado, no encontraba la carpeta datos. Ese codigo iba mucho más de mi conocimiento, y le pedi ayuda a arreglar todo esto pero para Disciplinas.

Respuesta: Parte 1. Las ayudas del final (leer, escribir, buscarX)

leer devuelve el contenido del archivo como JSONArray. Es el método que ya tienes. Tiene tres reglas:

Si el archivo no existe, devuelves un arreglo vacío (new JSONArray()), no un error. Así la primera ejecución arranca sin datos.

Si existe, lo lees entero con Files.readString(...).

Ese texto lo conviertes con new JSONArray(texto).

java

```
private JSONArray leer(String nombreArchivo) throws Exception {
    File archivo = new File(carpeta, nombreArchivo);
    if (!archivo.exists()) {
        return new JSONArray();
    }
    return new JSONArray(Files.readString(archivo.toPath()));
}
```

escribir hace lo contrario: crea la carpeta si no está y escribe el arreglo con sangría.

java

```
private void escribir(String nombreArchivo, JSONArray arreglo) throws Exception {
    Files.createDirectories(carpeta.toPath());
    Files.writeString(new File(carpeta, nombreArchivo).toPath(), arreglo.toString(2));
}
```

Las buscarX son un for que devuelve el objeto o null. Las necesitas cuando un archivo apunta a algo que ya cargaste y lo tienes que encontrar por su llave (nombre, id):

java

```
private DisciplinaIndividual buscarDisciplina(ArrayList<DisciplinaIndividual> lista, String nombre) {
    for (DisciplinaIndividual d : lista) {
        if (d.getNombre().equals(nombre)) {
            return d;
        }
    }
    return null;
}
```

Una por tipo de cosa que busques: buscarDeporte, buscarEquipo, buscarDisciplina. Para usuarios no hace falta, porque ya llegan en un Map y usas usuarios.get(login).

Cómo recorres un JSONArray: no es un ArrayList, no sirve el for-each. Siempre:

```
java
for (int i = 0; i < arreglo.length(); i++) {
    JSONObject json = arreglo.getJSONObject(i);   // objeto en la posición i
    // para un arreglo de textos: arreglo.getString(i)
}
```

Parte 2. Disciplinas: cómo pensarlo

Una disciplina contiene cosas que contienen cosas. Dibújalo antes de escribir:

```
Disciplina (nombre, maxInstalacionesSimultaneas)
 ├─ inscripciones[]   → (socio=login, nivel, puntajeRanking)
 └─ instalaciones[]   → (id, nombre, capacidad, techada, tipoCancha)
      └─ reservas[]   → (socio=login, fecha, horaInicio, horaFin, cantidad, modalidad)
           └─ invitados[] → logins
```

Regla: cada nivel es un JSONArray dentro del JSONObject del nivel de arriba, y cada nivel necesita su propio for. Usa una variable distinta por nivel (i, j, k, m) para no pisarte.

Guardar

Estructura del método (los puntos suspensivos los completas tú):

java

```
@Override
public void guardarDisciplinas(Collection<DisciplinaIndividual> disciplinas) throws Exception {
    JSONArray arreglo = new JSONArray();
    for (DisciplinaIndividual d : disciplinas) {
        JSONObject jd = new JSONObject();
        jd.put("nombre", d.getNombre());
        jd.put("maxInstalacionesSimultaneas", d.getMaxInstalacionesSimultaneas());

        // Nivel 2: inscripciones
        JSONArray inscripciones = new JSONArray();
        for (InscripcionDisciplina i : d.getInscripciones()) {
            JSONObject ji = new JSONObject();
            ji.put("socio", i.getSocio().getLogin());   // SOLO la llave, no el objeto
            // ... nivel y puntajeRanking
            inscripciones.put(ji);
        }
        jd.put("inscripciones", inscripciones);

        // Nivel 2: instalaciones
        JSONArray instalaciones = new JSONArray();
        for (Instalacion inst : d.getInstalaciones()) {
            JSONObject jin = new JSONObject();
            // ... id, nombre, capacidad, techada, tipoCancha

            // Nivel 3: reservas dentro de la instalación
            JSONArray reservas = new JSONArray();
            for (Reserva r : inst.getReservas()) {
                JSONObject jr = new JSONObject();
                jr.put("socio", r.getSocio().getLogin());
```

```
            jr.put("fecha", r.getFecha().toString());   // las fechas siempre con toString()
            // ... horaInicio, horaFin, cantidadJugadores, modalidad

            // Nivel 4: invitados (arreglo de logins)
            JSONArray invitados = new JSONArray();
            for (Socio inv : r.getInvitados()) {
               invitados.put(inv.getLogin());
            }
            jr.put("invitados", invitados);
            reservas.put(jr);
         }
         jin.put("reservas", reservas);
         instalaciones.put(jin);
      }
      jd.put("instalaciones", instalaciones);
      arreglo.put(jd);
   }
   escribir("disciplinas.json", arreglo);
}
```

El truco: cada nivel se cierra con .put(...) hacia arriba: la inscripción en inscripciones, inscripciones en jd, jd en arreglo.

Cargar

Es el espejo: recibe usuarios (porque inscripciones y reservas apuntan a socios) y reconstruye de afuera hacia adentro.

java

```
@Override
public ArrayList<DisciplinaIndividual> cargarDisciplinas(Map<String, Usuario> usuarios) throws Exception {
   ArrayList<DisciplinaIndividual> disciplinas = new ArrayList<DisciplinaIndividual>();
   JSONArray arreglo = leer("disciplinas.json");
```

```
    for (int i = 0; i < arreglo.length(); i++) {
        JSONObject jd = arreglo.getJSONObject(i);
        DisciplinaIndividual disciplina =
                                                new    DisciplinaIndividual(jd.getString("nombre"),
jd.getInt("maxInstalacionesSimultaneas"));

        JSONArray inscripciones = jd.getJSONArray("inscripciones");
        for (int j = 0; j < inscripciones.length(); j++) {
            JSONObject ji = inscripciones.getJSONObject(j);
            Socio socio = (Socio) usuarios.get(ji.getString("socio"));   // llave → objeto
            // disciplina.agregarInscripcion(new InscripcionDisciplina(socio, nivel, puntaje));
        }

        JSONArray instalaciones = jd.getJSONArray("instalaciones");
        for (int j = 0; j < instalaciones.length(); j++) {
            JSONObject jin = instalaciones.getJSONObject(j);
                    Instalacion instalacion = new Instalacion(/* id, nombre, capacidad, techada,
tipoCancha */);

            JSONArray reservas = jin.getJSONArray("reservas");
            for (int k = 0; k < reservas.length(); k++) {
                JSONObject jr = reservas.getJSONObject(k);
                 // Reserva reserva = new Reserva(socio, LocalDate.parse(...), LocalTime.parse(...),
...);

                JSONArray invitados = jr.getJSONArray("invitados");
                for (int m = 0; m < invitados.length(); m++) {
                    // reserva.agregarInvitado((Socio) usuarios.get(invitados.getString(m)));
                }
                // instalacion.agregarReserva(reserva);
            }
            // disciplina.agregarInstalacion(instalacion);
        }
```

```
        disciplinas.add(disciplina);
    }
    return disciplinas;
}
```

Reglas de lectura que te van a salvar:

getString, getInt, getDouble, getBoolean para lo simple.
getJSONArray("x") para bajar de nivel. getJSONObject(j) para el elemento j.
Las fechas vuelven con LocalDate.parse(...), LocalTime.parse(...).
Si te sale error, mira esto primero
JSONException: ... not found: escribiste un nombre de campo en guardar y otro en cargar. Deben ser idénticos, letra por letra.
ClassCastException: el login que buscas existe pero es otro tipo de usuario (por ejemplo, un Entrenador y no un Socio).
NullPointerException al cargar: usuarios.get(login) devolvió null. Cargaste las disciplinas antes que los usuarios, o el login no está en usuarios.json.
The method X is undefined: olvidaste un import. Ctrl+Shift+O.
Archivo vacío o raro: abre disciplinas.json y míralo. Si el guardado está bien, se ve el árbol anidado.

Decisiones: Con esta respuesta de base se comenzó a solucionar el problema, y cada vez que se avanzaba se pedía retroalimentación, lo mismo cuando me quedaba trabado. Pero toda la idea se saco mas que todo de este prompt y media plantilla, la cual de por si ya resolvía mucho del código por mi, pero se tenia que aplicar al resto de los casos restantes, y para sus respectivos gestores y que funcionen.

Miguel S.

Herramienta: Claude (Anthropic, 2026)

Documento de diseño.

Resumen: Utilicé la IA para guíar el proceso de identificación de las posibles funcionalidades críticas y sus posibles clases implicadas. Todo esto me permitió crear una especie de debate

donde se analizaron múltiples métodos con tal de identificar su complejidad e importancia dentro del sistema.

Prompt: Se le compartió a la IA los RF y RFN que se habían venido trabajando, junto con los literales del 1 al 4 para darle contexto. Seguido se le envió el siguiente PROMPT:
"Se tienen los siguientes requerimientos, son los que finalmente se van a dejar en la entrega final. Tengo que elegir las funcionalidades críticas donde se me pide elegir aproximadamente los 4 requerimientos funcionales más importantes. Dame una guía de cómo decidir cuáles funcionalidades críticas debo elegir."
Resultado: Se tuvo un flujo de conversación con la IA donde esta dio 3 criterios para evaluar los RF:
"**Tres criterios para evaluar cada RF**

Criterio	Qué significa
Complejidad de validación	¿Cuántas reglas de negocio distintas verifica antes de ejecutarse?
Clases involucradas	¿Cuántos objetos colaboran para completarla?
Centralidad	¿Es el corazón de un módulo del sistema, o un detalle secundario?

"
Con la respuesta dada por la IA se mantuvo una conversación donde se le fueron arrojando múltiples propuestas de posibles funcionalidades críticas, estas mismas venían acompañadas de las complejidades identificadas y a su vez de la importancia que tenían estas, la IA funcionó como guía para filtrar las que se consideran más pertinentes.
Decisión: Según las sugerencias de la IA, se eligieron como funcionalidades críticas los métodos que tenían mayor congruencia con los criterios de evaluación propuestos por la IA, donde se prioriza como criterio la complejidad de cada método.

Tulio R.

Herramienta: Claude (Anthropic, 2026)
Implementación JAVA

Resumen: Mi uso de AI se enfocó en asegurar que la implementación de la lógica y la consola fuera coherente con los requerimientos funcionales que queríamos cumplir. Tambien lo use para retroalimentación de la sintaxis del código para asegurar que implemente nuestro diseño en JAVA de forma correcta y sin funciones redundantes.

Prompt: Se le pasaron los RF's del documento de análisis y el UML detallado. "estos son los requerimientos y la estructura esperada del programa, asegurate que la implementación cumple con el diseño esperado y que responda a los requerimientos funcionales. Además asegura que la sintaxis de Java esté correcta y que no haya dejado errores de estructura."

Resultado: Resultó una conversación donde la AI resaltó varios requerimientos que no cumplimos con sugerencias para cómo solucionar lo que faltaba. Además la AI también me mostró los casos donde cometí errores de sintaxis y formato dentro del código.

Decisión: Varios de los cambios sugeridos fueron implementados y use el retorno de los errores de implementación de código para corregir mis errores y verificar el funcionamiento.

Implementación Consola

Prompt: Se le pasaron los RF's del documento de análisis y el funcionamiento esperado de la consola. "estos son los requerimientos y el funcionamiento esperado de la consola, cual es la forma más eficiente de implementar el funcionamiento"

Resultado: Resultó una conversación donde la AI me explico una implementación de un sistema lambda que permite que la consola delegue casos de prueba a métodos auxiliares que procesan los datos y retorna el resultado.

Decisión: Implemente el sistema recomendado y usa varios ejemplos de código generados por la AI para desarrollar el sistema dentro de nuestro proyecto.